



Building DIFT Systems for Software Security

Michael Dalton

Computer Systems Laboratory
Stanford University



Research Focus

- My focus is on **software attacks**
 - Protect apps from malicious input
 - Buffer overflows, XSS, SQL Injection, etc.
- Privacy, Crypto outside of scope
 - Information leaks, covert channels, etc.
- Assume software is **vulnerable**
 - But not malicious (no DRM/Malware/etc)



The Computer Security Crisis

- More systems are online, vulnerable
 - Banking, Power, Water, Government
- Threats have multiplied
 - XSS, SQL Injection, XSRF, Phishing, ...
- Old challenges remain
 - Buffer overflows, broken access control, authentication flaws



A Blast from the Past?

The New York Times
nytimes.com

PRINTER-FRIENDLY FORMAT
SPONSORED BY WATCH TRAILER

October 23, 2008

Attack code for critical Microsoft bug surfaces

By ROBERT MCMILLAN, [IDG](#)

Just hours after [Microsoft](#) posted details of a critical Windows bug, new attack code that exploits the flaw has surfaced.

It took developers of the [Immunity](#) security testing tool two hours to [write](#) their exploit, after Microsoft released a patch for the issue Thursday morning. Software developed by Immunity is made available only to paying customers, which means that not everyone has access to the new attack, but security experts expect that some version of the code will begin circulating in public very soon.

Microsoft took the unusual step of rushing out an emergency patch for the flaw Thursday, two weeks after noticing a small number of targeted attacks that exploited the bug.

The vulnerability was not publicly known before Thursday; however, by issuing its patch, Microsoft has given hackers and security researchers enough information to develop their own attack code.

The flaw lies in the Windows Server service, used to connect different network resources such as file and print servers over a network. By sending malicious messages to a Windows machine that uses Windows Server, an attacker could take control of the computer, Microsoft said.

Apparently, it doesn't take much effort to write this type of attack code.

"It is very exploitable," said Immunity Security Researcher Bas Albert. "It's a very controllable stack overflow."

Stack overflow bugs are caused when a programming error allows the attacker to write a command on parts of the computer's memory that would normally be out of limits and then cause that command to be run by the victim's computer.



Wave of the Future?

October 13th, 2008

The Image Group Website Hacked Through SQL-Injection, Credit Cards Data Stolen

From January to August 2008, hackers through an SQL injection flaw were able to access names and credit or debit card information of the persons who placed orders on The Image Group e-commerce website. The Image Group (<http://www.theimagegroup.net>) is a firm for promotional products and corporate merchandise headquartered in Ohio.

The Image Group has notified the New Hampshire State Attorney General and online customers that their e-commerce site fell victim to a series of successful SQL injection attacks. The compromised database contained sensitive personal and financial information belonging to customers of the company.

While this was discovered in August, it appears that the unauthorized access began in January and occurred again in August of this year. Names, credit cards/debit cards numbers, expiration dates, addresses and the CVV codes were accessed by hackers. No social security numbers or dates of birth were involved.

Upon learning of the breach, the firm shut down the web site through which the unauthorized access occurred. In addition, they had a forensic audit performed. Currently they are working with the merchant bank and the Card Associations to address issues associated with the credit card information taken and to notify the issuing banks for those cards.

It seems that the website domain related to this incident is theideacatalog.com, for which the registrant and administrative contact is Target Marketing. The IP address for the www.ideacatalog.com website is 74.84.205.104 and belongs to the block 74.84.205.0 - 74.84.205.255, which is assigned to Target Marketing. The site may have been developed and/or managed by Target Marketing.

Toll-free number for questions is 866-272-5162.

Source: cyberinsecure.com



Secure Programming is Hard

- **Validate** untrusted data before using it
 - Apply correct validation for each possible vuln
- **Safety requires perfect code**
 - Miss or incorrectly perform check  vulnerable
- **New languages will not save us**
 - Don't help with existing binaries
 - Lots of development still using C and C++
 - Java, Lisp still vulnerable to XSS, SQL Inj,...





Ideal Security Platform

- Well-defined abstraction
 - Applicable to many security problems
- Efficient implementation
- Practical
 - Does not require source code or app changes
- Robust policies
 - No false positives, false negatives



Why haven't we solved this?

- Existing solutions incomplete
 - Stack canaries, heap red zones, NX, ASLR
 - **Not robust, incompatible**
 - Web App Firewall, IDS
 - **Not robust** (heuristic)
- Problem
 - All solutions are **ad-hoc**, not general
 - Many based on heuristics of **attacker data**
 - Attackers adapt



Thesis Overview

- Use Dynamic Information Flow Tracking (DIFT) as the one abstraction to build security defenses

- Thesis contributions
 - Co-developed a **flexible** hardware design for **efficient, practical** DIFT on binaries
 - Including a **real** full-system prototype (HW+SW)
 - Developed novel **robust** DIFT Policies
 - **First** buffer overflow protection policy protecting both userspace and kernelspace
 - **First** authentication/authorization bypass policy protecting web applications



Outline

- DIFT overview
- Raksha: hardware support for DIFT [WDDD'06, ISCA'07]
 - Flexible HW design for efficient, practical DIFT on binaries
- DIFT policies for buffer overflow protection [USENIX Security'08]
 - Protection for userspace & kernel space without false positives
- DIFT policies for web application vulnerabilities [USENIX Security '09]
 - Protection against authentication & access control attacks

DIFT: Dynamic Information Flow Tracking



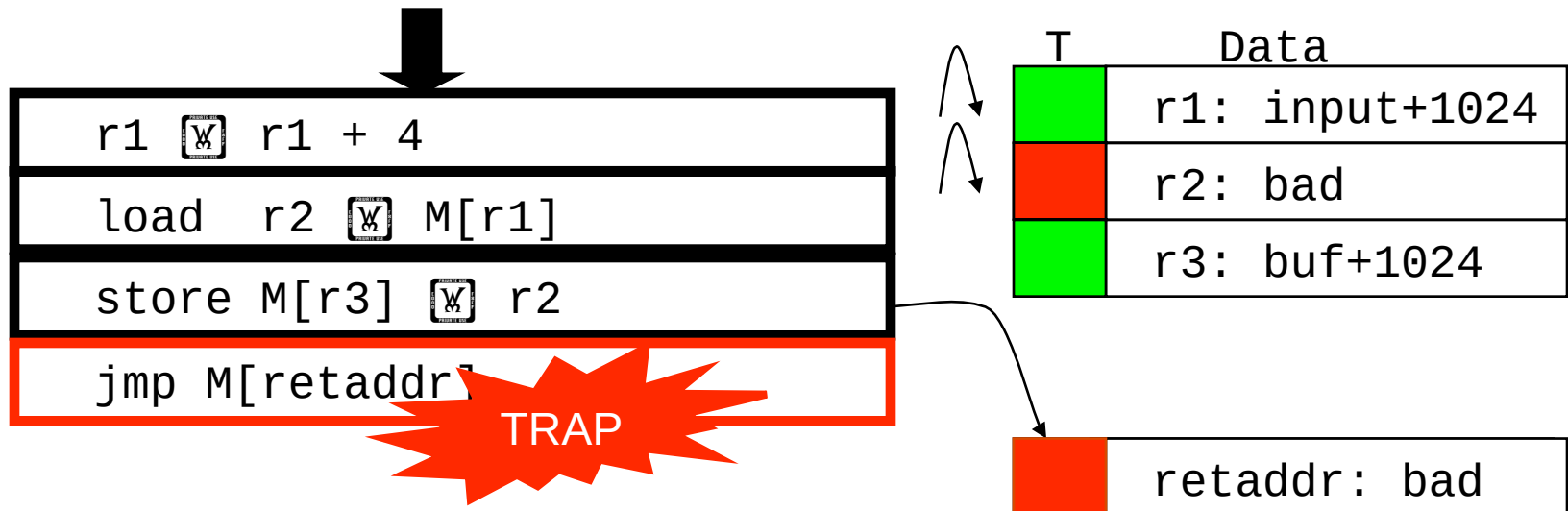
- DIFT taints data from untrusted sources
 - Extra tag bit per word marks if untrusted
- Propagate taint during program execution
 - Operations with tainted data produce tainted results
- Check for unsafe uses of tainted data
 - Tainted code execution
 - Tainted pointer dereference (code & data)
 - Tainted SQL command
- Can detect both low-level & high-level threats

DIFT Example: Memory Corruption



Vulnerable C Code

```
char buf[1024];  
strcpy(buf, input); //buffer overflow
```



Tainted pointer dereference  security trap



DIFT Example: SQL Injection

Username:
Password:

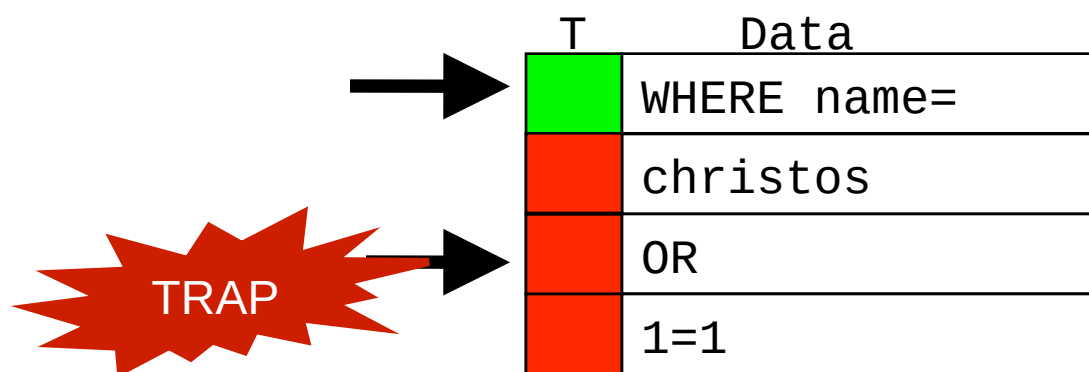
christos' OR '1'='1'



Vulnerable SQL Code

```
SELECT * FROM table
```

```
WHERE name= 'username' OR '1'='1';
```



Tainted SQL command ↴ security trap



Implementing DIFT on Binaries

- **Software DIFT** [Newsome'05, Quin'06]
 - Use Dynamic Binary Translation (DBT) to implement DIFT
 - ☑ Runs on existing hardware, flexible security policies
 - ☒ **High overheads** (3–40x), **incompatible** with threaded or self-modifying code, limited to a single core
- **Hardware DIFT** [Suh'04, Crandall'04, Chen'05]
 - Modify CPU caches, registers, memory consistency, DRAM
 - ☑ Negligible overhead, works for all types of binaries, multi-core
 - ☒ **Inflexible** policies (false positives/negatives), cannot protect OS
- **Best of both worlds**
 - HW for tag propagation and checks
 - SW for policy management and high-level analysis
 - Robust, flexible, practical, end-to-end, and fast

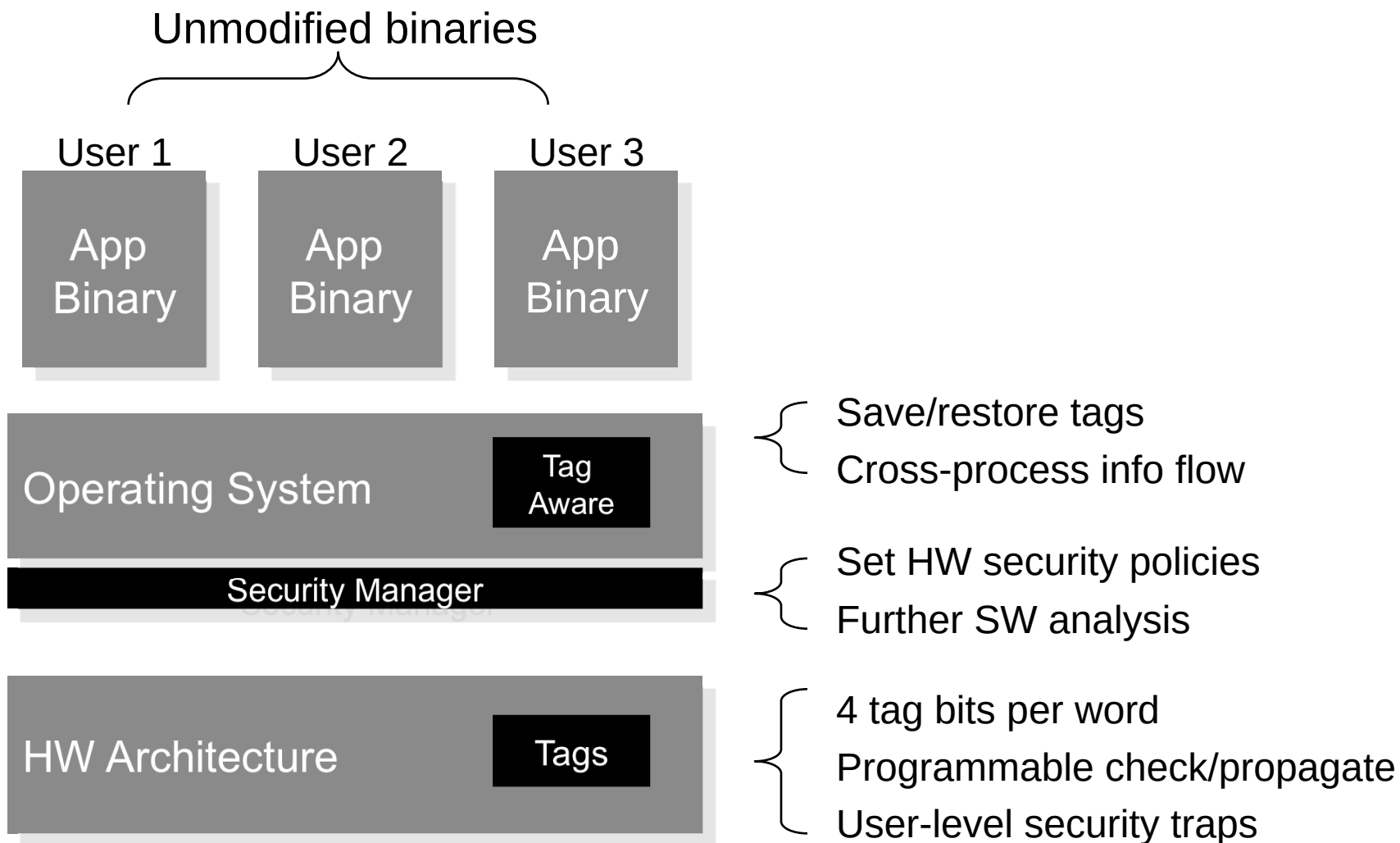


Outline

- DIFT overview
- Raksha: hardware support for DIFT [WDDD'06, ISCA'07]
 - Flexible HW design for efficient, practical DIFT on binaries
- DIFT policies for buffer overflow protection [USENIX Security'08]
 - Protection for userspace & kernel space without false positives
- DIFT policies for web application vulnerabilities [USENIX Security '09]
 - Protection against authentication & access control attacks

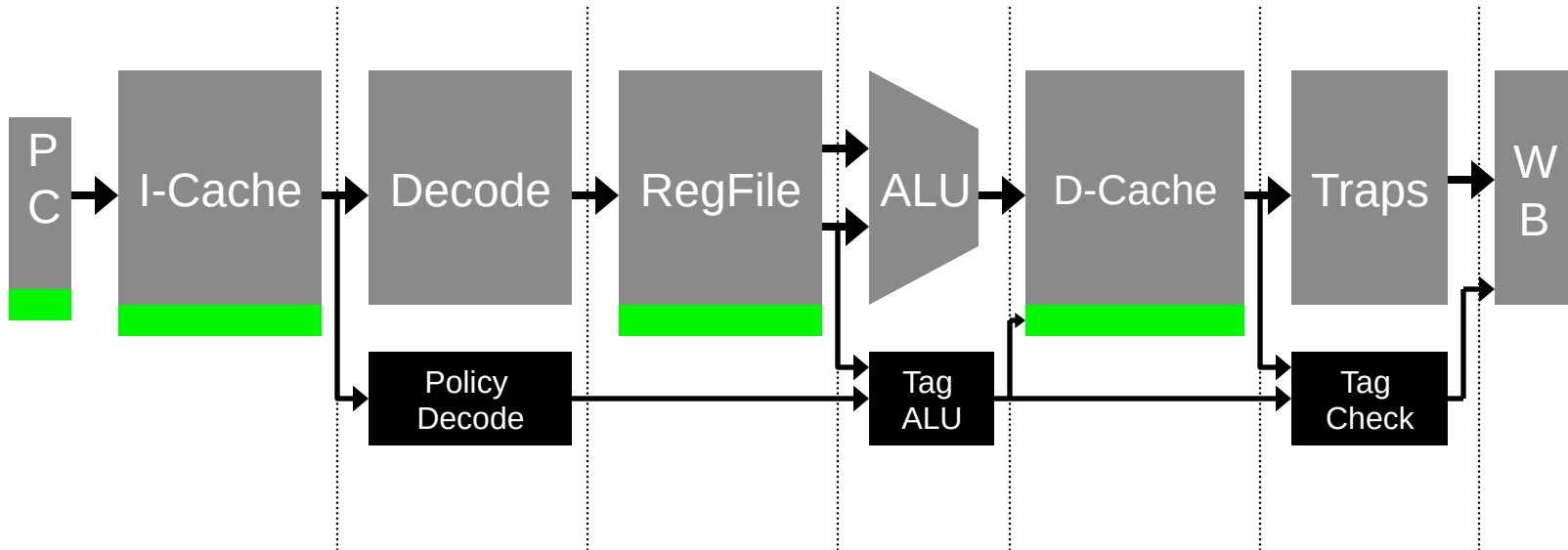


Raksha System Overview





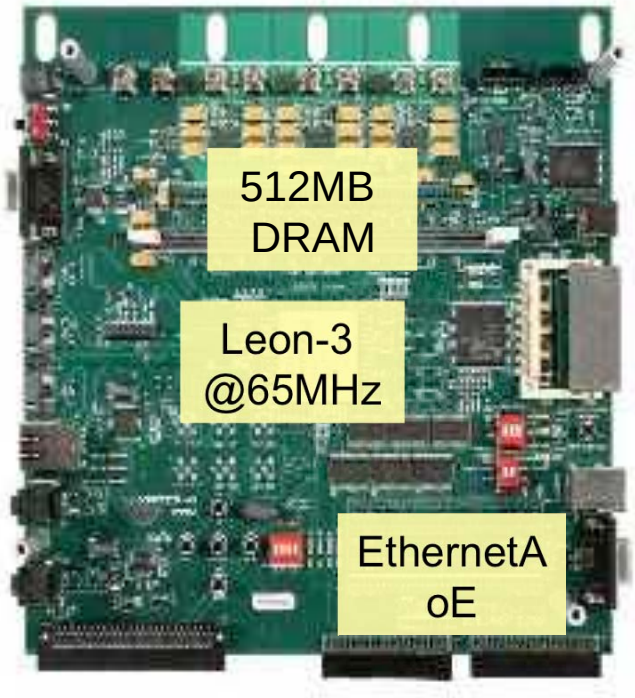
Raksha Hardware



- Registers & memory extended with tag bits
 - See Hari Kannan's thesis for efficient, multi-granularity tag store
- Tags flow through pipeline along with corresponding data
 - No changes in forwarding logic



Raksha Prototype



■ Hardware

- Modified SPARC V8 CPU (LEON-3)
- Mapped to FPGA board

■ Software

- Full-featured Gentoo Linux workstation
- Used with >14k packages (LAMP, etc)

■ Design statistics

- Clock frequency: same as original
- Logic: +7% overhead
- Performance: <1% slowdown
 - Across a wide range of applications
 - SW DIFT is 3-40x slowdown



HW/SW Interface for DIFT Policies

- A pair of policy registers per tag bit
 - Set by security manager (SW) when and as needed
- Policy granularity: operation type
 - Select input operands to check if tainted
 - Select input operands that propagate taint to output
 - Select the propagation mode (and, or, xor)
- ISA instructions decomposed to ≥ 1 operations
 - Types: ALU, comparison, insn fetch, data movement, ...
 - Makes policies independent of ISA packaging
 - Same HW policies for both RISC & CISC ISAs
 - Don't care how operations are packaged into ISA insns

Propagate Policy Example: Load



load r2 ← M[r1+offset]

Propagate Enables

1. Propagate only from source register

Tag(r2) $\leftarrow$ Tag(r1)

2. Propagate only from source address

Tag(r2) $\leftarrow$ Tag(M[r1+offset])

3. Propagate only from both sources

OR mode: Tag(r2) $\leftarrow$ Tag(r1) | Tag(M[r1+offset])

AND mode: Tag(r2) $\leftarrow$ Tag(r1) & Tag(M[r1+offset])

XOR mode: Tag(r2) $\leftarrow$ Tag(r1) ^ Tag(M[r1+offset])



Check Policy Example: Load

load $r2 \leftarrow M[r1+offset]$

Check Enables

1. Check source register

 If $\text{Tag}(r1) == 1$ then security_trap

2. Check source address

 If $\text{Tag}(M[r1+offset]) == 1$ then security_trap

Both enables may be set simultaneously

Support for checks across multiple tag bits



Security Policies Overview

- Complete set of DIFT policies used
 1. Pointer-injection based policy (covered later)
 - Robust protection against BOF
 2. Bounds check based policy
 - Limited to control pointer only (no false positives)
 - Detects offset-based attacks for control pointers
 3. Red zone policy
 - Use tags to sandbox heap metadata
 - Detects offset-based attacks on heap data
 4. Format string policy
 - Checks tainted arguments to print commands
 5. Policy for high-level attacks
 - SQL and command injection, XSS, directory traversal
 6. Sandboxing policy to protect security monitor
- Mapped to 4 tag bits due to commonalities



Security Experiments

Program	Lang.	Attack	Detected Vulnerability
tracert	C	Double Free	Tainted data ptr
polymorph	C	Buffer Overflow	Tainted code ptr
Wu-FTPD	C	Format String	Tainted '%n' in fprintf string
gzip	C	Directory Traversal	Open tainted dir
Wabbit	PHP	Directory Traversal	Escape Apache root w. tainted '..'
OpenSSH	C	Command Injection	Execve tainted file
ProFTPD	C	SQL Injection	Tainted SQL command
htdig	C++	Cross-site Scripting	Tainted <script> tag
PhpSysInfo	PHP	Cross-site Scripting	Tainted <script> tag
Scry	PHP	Cross-site Scripting	Tainted <script> tag

- **Unmodified SPARC binaries from real-world programs**
 - Basic/net utilities, servers, web apps, search engine



Security Experiments

Program	Lang.	Attack	Detected Vulnerability
traceroute	C	Double Free	Tainted data ptr
polymorph	C	Buffer Overflow	Tainted code ptr
Wu-FTPD	C	Format String	Tainted '%n' in vfprintf string
gzip	C	Directory Traversal	Open tainted dir
Wabbit	PHP	Directory Traversal	Escape Apache root w. tainted '..'
OpenSSH	C	Command Injection	Execve tainted file
ProFTPD	C	SQL Injection	Tainted SQL command
htdig	C++	Cross-site Scripting	Tainted <script> tag
PhpSysInfo	PHP	Cross-site Scripting	Tainted <script> tag
Scry	PHP	Cross-site Scripting	Tainted <script> tag

- Protection against low-level memory corruptions
 - Both control & non-control data attacks



Security Experiments

Program	Lang.	Attack	Detected Vulnerability
traceroute	C	Double Free	Tainted data ptr
polymorph	C	Buffer Overflow	Tainted code ptr
Wu-FTPD	C	Format String	Tainted '%n' in vfprintf string
gzip	C	Directory Traversal	Open tainted dir
Wabbit	PHP	Directory Traversal	Escape Apache root w. tainted '..'
OpenSSH	C	Command Injection	Execve tainted file
ProFTPD	C	SQL Injection	Tainted SQL command
htdig	C++	Cross-site Scripting	Tainted <script> tag
PhpSysInfo	PHP	Cross-site Scripting	Tainted <script> tag
Scry	PHP	Cross-site Scripting	Tainted <script> tag

- 1st DIFT system to detect high-level attacks
 - Without need to recompile applications



Security Experiments

Program	Lang.	Attack	Detected Vulnerability
traceroute	C	Double Free	Tainted data ptr
polymorph	C	Buffer Overflow	Tainted code ptr
Wu-FTPD	C	Format String	Tainted '%n' in vfprintf string
gzip	C	Directory Traversal	Open tainted dir
Wabbit	PHP	Directory Traversal	Escape Apache root w. tainted '..'
OpenSSH	C	Command Injection	Execve tainted file
ProFTPD	C	SQL Injection	Tainted SQL command
htdig	C++	Cross-site Scripting	Tainted <script> tag
PhpSysInfo	PHP	Cross-site Scripting	Tainted <script> tag
Scry	PHP	Cross-site Scripting	Tainted <script> tag

- Protection is independent of programming language
 - Propagation & checks at the level of basic ops



Outline

- DIFT overview
- Raksha: hardware support for DIFT [WDDD'06, ISCA'07]
 - Flexible HW design for efficient, practical DIFT on binaries
- DIFT policies for buffer overflow protection [USENIX Security'08]
 - Protection for userspace & kernel space without false positives
- DIFT policies for web application vulnerabilities [USENIX Security '09]
 - Protection against authentication & access control attacks



The Challenge of BOF Protection

- Must recognize when app validates its input
 - In order to untaint validated data
 - Conservative untainting  false positives
 - Liberal untainting  false negatives

- Original approach from previous work
 - Assume comparisons used for bounds checks
 - Untaint on comparison



Bounds Check Flaws

- Hard to identify all bounds check operations
 - `*str++ = digits[val % 10]` (glibc)
 - `ent = hashtable[x & TABLESZ - 1]` (GCC)
- Not all comparisons are bounds checks
 - `if (chunksz(sz) < FASTBIN_SZ)` (malloc)
 - Caused a false negative in traceroute exploit
- Apps validate without bounds checks
 - `return isdigit[(unsigned char)x]` (glibc)
 - Only safe if we know array has 256 entries



BOF Prevention Insight

- Bounds check recognition fatally flawed
 - False positives on most real-world apps
- New approach: pointer injection
 - Don't try to recognize bounds checks
 - Buffer overflows **overwrite pointers**
- Pointers critical to BOF attacks
 - Stack: frame ptr, return addr, function ptr
 - Heap: malloc headers, vtable, exception hdl



Pointer Injection Overview

- Rely on real-world safe pointer usage
 - Tainted data used as pointer **index**
 - Never as pointer **base address**
- Prevent pointer injection
 - Only dereference pointer values **from app**
 - User input must be combined w/ app ptr
 - Index/offset protected by other policies
- End result: never deref untrusted ptr



Pointer Injection DIFT Policy

- **Tag Management (2 tag bits/word)**
 - **Pointer bit** – Set for legitimate app pointers
 - **Taint bit** – Set for untrusted data
- **Propagate tags at runtime**
 - T-bit on all data ops, P-bit on pointer arith
- **Check on indirect jump/load/store**
 - Failure if address has T-bit set, P-bit clear



Identifying Pointers

- Only have to identify **root pointers**
 - Pointers not derived from another pointer
 - All other pointers safely handled via tag prop
- Initialize P-bit for dynamic alloc'd mem
 - Mem alloc syscall return value has P-bit set
- Runtime propagation on pointer arith
 - +, -, &, etc.



Identifying Static Pointers

- Relocation makes identification practical
 - Relocation info isn't present in exec/lib, but...
 - **Very few ELF/PE relocation entry types**
- Scan all object files (exec+libs) at startup
 - Identify refs to statically alloc'd mem that comply with object file relocation entry types
 - Set P-bit on **insns**, **data** referring to static mem
 - Scan any dynamically loaded libs at runtime



Userspace BOF Results

Program	Attack	Detection
Polymorph	Stack overflow	Tainted code ptr
Atphhttpd	Stack overflow	Tainted code ptr
Nullhttpd	Heap overflow	Tainted data ptr
Traceroute	Double free	Tainted data ptr
Sendmail	BSS overflow	Tainted data ptr

- All applications are unmodified binaries
- Protection enabled for all of userspace
- No false positives



Kernelspace BOF Results

Module	Attack	Detection
Quotactl syscall	User/Kernel Pointer	User pointer to OS data
I2o driver	User/Kernel Pointer	User pointer to OS data
Sendmsg syscall	Stack, Heap Overflow	Tainted data pointer
Moxa driver	BSS Overflow	Tainted data pointer
Cm4040 driver	Heap Overflow	Tainted data pointer

- Protection enabled for all of kernelspace
- No false positives (found one kernel bug!)

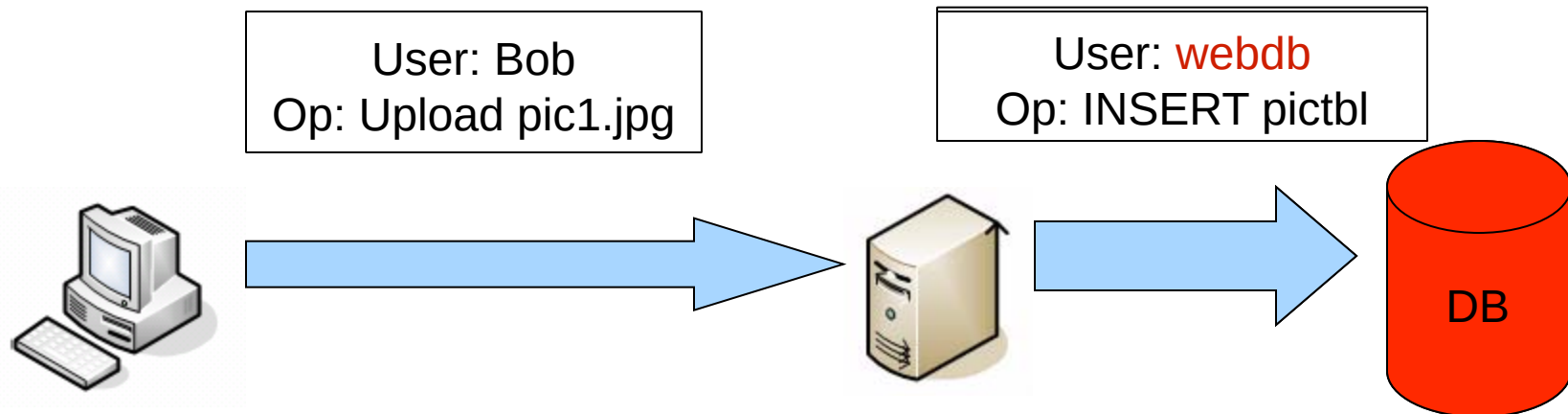


Outline

- DIFT overview
- Raksha: hardware support for DIFT [WDDD'06, ISCA'07]
 - Flexible HW design for efficient, practical DIFT on binaries
- DIFT policies for buffer overflow protection [USENIX Security'08]
 - Protection for userspace & kernel space without false positives
- **DIFT policies for web application vulnerabilities** [USENIX Security '09]
 - Protection against authentication & access control attacks



Web Application Overview





Web Authentication is Broken

- **Semantic Gap** – independent auth sys
 - **Web Authentication**
 - **System Authentication** (DB, FS, etc.)
- Webapps are effectively setuid progs
 - All FS, DB ops have privs of webapp
 - Not privs of webapp **user** (Confused Deputy)
- Safe **only** if webapp inserts checks
 - Check web app user before **all** FS/DB op



Authorization Bypass

- Resource access without authorization
 - Missing authorization check
 - Incorrect authorization check

```
if(client_authorized($_GET['fileName']))  
    openFile($_GET['filename']))
```

Add URL parameter: filename=/etc/passwd



Authentication Bypass

- Authentication without valid credentials
 - URL/Cookie Validation Error
 - Weak Crypto
 - Etc

```
if (isset($_COOKIE['user']))  
    $userName = $_COOKIE['user'];
```

Edit cookie, add name/value pair: 'user=admin'

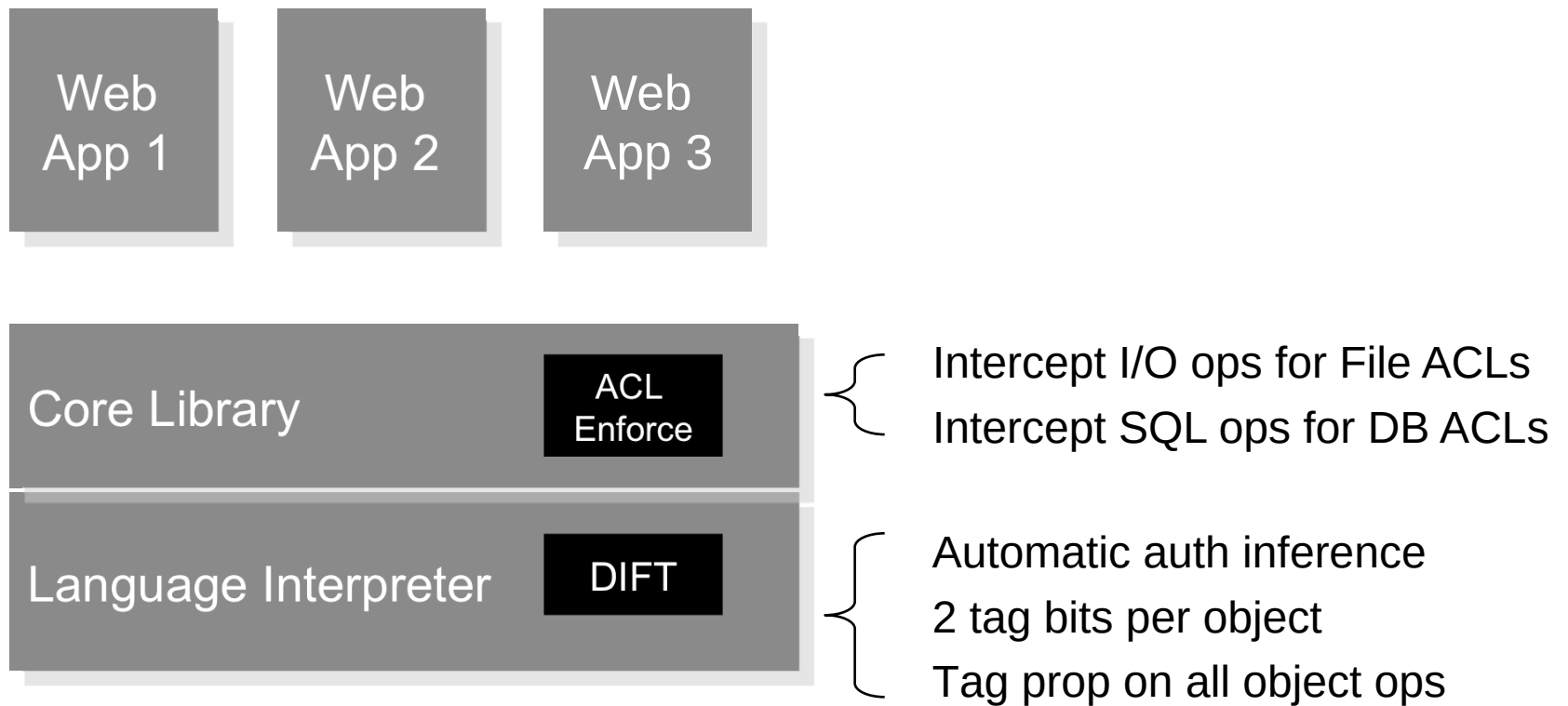


Nemesis

- A DIFT-based system that
- **Stops** authentication, authorization atks
 - Without requiring app auth code rewrites
- **Infers** when authentication done **safely**
 - Use **DIFT** to track auth credentials
- **Enforces** ACLs automatically on file/DB
 - ACLs specify privs for web clients



Nemesis System Overview





Safe Authentication Inference

- Propagate user credential, taint bits
 - 2 tag bits per object (String, integer, etc)
- **Infer** when auth occurs safely
 - Tainted info compared equal to auth cred
 - Add check to string or array comparison op
- Record **authentication inferred** user
 - Auth bypass attacks do not change this user



Authentication Example

Application Code

```
$user = $_GET['username']  
$user = mysql_real_escape_string($user)  
$pw = md5sum($_GET['password'])  
  
$realpw = $db->query("SELECT pw FROM  
    users WHERE userName =" + $user +  
    ".")  
  
if ($pw == $realpw) {
```

Authenticated!

T	P	Variable
		\$user
		\$pw
		\$realpw



Authorization Enforcement

- Enforce ACLs on FS, DB access
 - Apply to **authentication inferred user**
- Restrict DB table/row, file access
 - Many tables store per-user rows
- Taint information used in some rules
 - New user registration
 - Password change



Nemesis Requirements

- Authentication inference
 - Table/column info for auth credentials

- ACL enforcement
 - ACL from sysadmin for DB, File access

- Future work
 - Log DB, File ops along with inferred user
 - Auto-generate ACLs from logs



Nemesis Prototype

- Added DIFT support to PHP interpreter
 - Password, Taint bits for String, int, etc
 - Assume Raksha checking OS & PHP interpreter for low-level attacks
- Auth inference on string comparison
 - ==, != operators
- Don't have a full SQL query rewriter
 - Had to manually insert DB checks

Experimental Results



Application	Size (Lines)	Auth Lines Added	ACL Check Lines Added	Attack Prevented
Php iCalendar	13,500	3	22	Auth Bypass
PhpStat	12,700	3	17	Missing ACL Check
Bilboblog	2,000	3	11	Incorrect ACL Check
phpFastNews	500	5	17	Auth Bypass
Linha Gallery	50,000	15	49	SQL Injection in Password Check
DeluxeBB	22,000	6	143	Missing ACL Check

No discernible performance overhead



Thesis Overview

- Use Dynamic Information Flow Tracking (DIFT) as the one abstraction to build security defenses

- Thesis contributions
 - Co-developed a **flexible** hardware design for **efficient, practical** DIFT on binaries
 - Including a **real** full-system prototype (HW+SW)
 - Developed novel **robust** DIFT Policies
 - **First** buffer overflow protection policy protecting both userspace and kernelspace
 - **First** authentication/authorization bypass policy protecting web applications



Conclusion

- DIFT is a promising security solution
 - Prevents HL/LL attacks, does not need src code
- Co-developed Raksha, a **flexible** hardware design for **efficient, practical** DIFT on binaries
 - Including a **real** full-system prototype (HW+SW)
- Developed novel **robust** DIFT Policies
 - **First** buffer overflow protection policy protecting both userspace and kernelspace
 - **First** authentication/authorization bypass policy protecting web applications



Bibliography

- "Deconstructing Hardware Architectures for Security," Michael Dalton, Hari Kannan, Christos Kozyrakis. *5th Annual Workshop on Duplicating, Deconstructing, and Debunking (WDDD) at ISCA*, Boston, MA, June 2006.
- "Raksha: A Flexible Information Flow Architecture for Software Security," Michael Dalton, Hari Kannan, Christos Kozyrakis. *Proceedings of the 34th Intl. Symposium on Computer Architecture (ISCA)*, San Diego, CA, June 2007.
- "Raksha: A Flexible Architecture for Software Security," Hari Kannan, Michael Dalton, Christos Kozyrakis. *Technical Record of the 19th Hot Chips Symposium*, Palo Alto, CA, August 2007.
- "Thread-Safe Dynamic Binary Translation Using Transactional Memory," JaeWoong Chung, Michael Dalton, Hari Kannan, Christos Kozyrakis. *Proceedings of the 14th Intl. Symposium on High-Performance Computer Architecture (HPCA)*, Salt Lake City, UT, February 2008.



Bibliography cont'd

- "Real-World Buffer Overflow Protection for Userspace and Kernel-space," Michael Dalton, Hari Kannan, Christos Kozyrakis. *Proceedings of the 17th Usenix Security Symposium*, San Jose, CA, July 2008.
- "Hardware Enforcement of Application Security Policies," Nickolai Zeldovich, Hari Kannan, Michael Dalton, Christos Kozyrakis. *Proceedings of the 8th Usenix Symposium on Operating Systems Design & Implementation (OSDI)*, San Diego, CA, December 2008
- "Decoupling Dynamic Information Flow Tracking with a Dedicated Coprocessor," Hari Kannan, Michael Dalton, Christos Kozyrakis. *Proceedings of the 39th Intl. Conference on Dependable Systems and Networks (DSN)*, Estoril, Portugal, June 2009.
- "Nemesis: Preventing Authentication and Access Control Vulnerabilities in Web Applications," Michael Dalton, Nickolai Zeldovich, Christos Kozyrakis, *Proceedings of the 18th Usenix Security Symposium*, Montreal, CA, August 2009.



Acknowledgements

- Family
- Friends
- Colleagues
- Christos
- Orals Committee